

Multi-Model Comparison Guide

Overview

The tangier trading strategy now supports **multi-model comparison**, allowing you to train and compare 5 different machine learning models simultaneously. The system automatically selects the best model for each cryptocurrency based on a composite scoring metric.

Models Included

- 1. **LightGBM** - Fast gradient boosting, excellent for tabular data
- 2. **XGBoost** - Robust gradient boosting with regularization
- 3. **Random Forest** - Ensemble method, resistant to overfitting
- 4. **Logistic Regression** - Classical baseline, interpretable
- 5. **Neural Network** - Deep learning, captures complex patterns

Composite Scoring Metric

Each model is scored using a weighted combination of three metrics:

Plain Text

Composite Score = (Return × 0.6) + (Sharpe Ratio × 0.3) + (Win Rate × 0.1)

Weights:

- **Return (60%)** - Primary objective: maximize profitability
- **Sharpe Ratio (30%)** - Risk-adjusted returns: balance profit with volatility
- **Win Rate (10%)** - Consistency: prefer strategies with higher win rates

This composite score balances profitability with risk management.

How to Use

Option 1: Run Multi-Model Comparison (Recommended)

Bash

```
python main_multi_model.py
```

This will:

1. Train all 5 models on all symbols
2. Backtest each model
3. Calculate composite scores
4. Select the best model per symbol
5. Generate plots for best models only
6. Export comparison results

Option 2: Run Single Model (Original)

Bash

```
python main.py
```

This runs the original single-model strategy using LightGBM.

Output Files

After running `python main_multi_model.py`, you'll get:

Results Files

- `model_comparison_results.csv` - Detailed comparison of all models
- `best_models_per_symbol.csv` - Best model for each symbol
- `SYMBOL_MODEL_price_signals.html` - Price chart with signals (best model)
- `SYMBOL_MODEL_equity_curve.html` - Equity curve (best model)

CSV Columns in `model_comparison_results.csv`

Column	Description
Symbol	Trading symbol (e.g., BTCUSDT)
Model	Model name (lgbm, xgboost, etc.)
Return (%)	Strategy return percentage

Sharpe Ratio	Risk-adjusted return metric
Win Rate (%)	Percentage of winning trades
Composite Score	Overall score (higher is better)
Num Trades	Number of trades executed
Avg Win (%)	Average winning trade percentage
Avg Loss (%)	Average losing trade percentage
Profit Factor	Ratio of gross profit to gross loss
Max Drawdown (%)	Maximum peak-to-trough decline
Buy & Hold (%)	Buy-and-hold benchmark return
Outperformance (%)	Strategy return - Buy & Hold return

Example Results

Plain Text

MODEL COMPARISON SUMMARY

=====

Best Overall Model: xgboost (BTCUSDT)

Composite Score: 0.8234

Return: 45.23%

Sharpe Ratio: 1.2345

Win Rate: 52.34%

Best Model per Symbol:

BTCUSDT: xgboost (Score: 0.8234, Return: 45.23%)

ETHUSDT: lgbm (Score: 0.7891, Return: 38.45%)

BNBUSDT: neural_network (Score: 0.7654, Return: 35.67%)

Model Performance Ranking (by average Composite Score):

xgboost: 0.7856 ± 0.0234 (n=3)

lgbm: 0.7623 ± 0.0345 (n=3)

neural_network: 0.7234 ± 0.0456 (n=3)

random_forest: 0.6945 ± 0.0567 (n=3)

logistic_regression: 0.6234 ± 0.0678 (n=3)

Interpreting Results

Composite Score

- **Higher is better** (0.0 to 1.0+)
- Score > 0.7 = Excellent model
- Score 0.5-0.7 = Good model
- Score < 0.5 = Poor model

Return vs Sharpe Ratio

- **High Return, Low Sharpe:** Risky, inconsistent
- **High Return, High Sharpe:** Ideal (profitable and stable)
- **Low Return, High Sharpe:** Safe but not profitable

Win Rate

- **> 50%:** More winning trades than losing
- **40-50%:** Balanced, relies on profit factor
- **< 40%:** Fewer wins, needs larger average wins

Customizing Models

To modify model parameters, edit `src/multi_model_trainer.py` :

Python

```
def _train_lgbm(self, X: np.ndarray, y: np.ndarray):  
    """Train LightGBM model."""  
    model = lgb.LGBMClassifier(  
        n_estimators=100,      # Number of trees  
        learning_rate=0.1,    # Learning rate  
        max_depth=7,          # Tree depth  
        # ... modify parameters here  
    )  
    model.fit(X, y)  
    return model
```

Customizing Scoring

To change the composite score weights, edit `src/model_comparator.py` :

Python

```
def calculate_composite_score(self, return_pct, sharpe_ratio, win_rate):  
    # Current weights: Return=60%, Sharpe=30%, Win Rate=10%  
    score = (return_normalized * 0.6) + (sharpe_normalized * 0.3) +  
(win_rate_normalized * 0.1)  
  
    # Example: Prioritize Sharpe Ratio (risk management)  
    # score = (return_normalized * 0.4) + (sharpe_normalized * 0.5) +  
(win_rate_normalized * 0.1)  
  
    return score
```

Performance Tips

1. **Training Time:** Multi-model comparison takes 5x longer than single model
 - Each symbol $\times$ 5 models = 5 backtests per symbol
 - Use fewer symbols for faster testing
2. **Memory Usage:** Neural networks use more memory
 - If running out of memory, remove `neural_network` from comparison
3. **Data Quality:** More data = better models
 - Use `LOOKBACK_PERIOD = "2 years ago"` for better training
4. **Feature Selection:** More features = slower training
 - Reduce number of indicators in `feature_engineer.py` if too slow

Troubleshooting

Error: "XGBoost not installed"

Bash

```
pip install xgboost
```

Error: "SMOTE failed"

- Occurs when too few samples in a class
- Solution: Increase `LOOKBACK_PERIOD` to get more data

All models have low scores

- Check if signals are being generated (should see Buy/Hold/Sell counts)
- Verify data quality and indicators
- Try adjusting `BUY_THRESHOLD` and `SELL_THRESHOLD` in config

Next Steps

1. Run multi-model comparison: `python main_multi_model.py`
2. Review `model_comparison_results.csv`
3. Identify best model for each symbol
4. Use best model for live trading
5. Retrain monthly with fresh data

Advanced Usage

Deploy Best Model to Live Trading

Python

```
from src.multi_model_trainer import MultiModelTrainer

trainer = MultiModelTrainer(config)
best_model, scaler = trainer.load_model('xgboost', 'BTCUSDT')

# Use in live trading
signals = trainer.predict_signals(best_model, X, scaler)
```

Compare Only Specific Models

Edit `main_multi_model.py` and modify:

Python

```
# Train only LightGBM and XGBoost
for model_name in ['lgbm', 'xgboost']:
    # ... training code
```

For more information, see `README.md` and `QUICKSTART.md`